

Day 122: REST APIs and HTTP Methods

1. Day number

Day 122

2. Topic name

REST, GET, POST, PUT, and DELETE

Today you will learn how APIs commonly use different HTTP methods for different kinds of operations.

3. Connection

Yesterday, on **Day 121**, you learned the basic HTTP flow:

```
Client
  ↓ request
Server
  ↓ response
Client
```

You learned that a client sends an HTTP request to a server, and the server sends back a response.

Today, you will add one important idea:

| Different HTTP methods usually represent different intentions.

For example:

```
GET    → read data
POST   → create data
PUT    → update data
DELETE → delete data
```

These ideas are commonly used when designing **REST APIs**.

4. Important topics

REST API

REST stands for:

Representational State Transfer

You do not need to memorize the full definition yet.

At a beginner level, think of a REST API as:

| An API that organizes data as resources and commonly uses HTTP methods to perform operations on those resources.

For example, an online store may have a product resource:

`/products`

and an individual product:

`/products/10`

A client might use different HTTP methods with those URLs.

GET `/products/10`

PUT `/products/10`

DELETE `/products/10`

The URL identifies the resource.

The HTTP method describes what the client wants to do with it.

Resource

A **resource** is something that the API manages.

Examples:

product

customer

order

book

student

employee

In a products API, one product could look like:

```
{  
  "id": 10,  
  "name": "Keyboard",  
  "price": 1200  
}
```

Here:

Product = resource

A collection of products might be represented by:

`/products`

A specific product might be represented by:

`/products/10`

Endpoint

An **endpoint** is a location in an API that a client can send requests to.

Example:

`https://api.example.com/products`

Another endpoint might be:

`https://api.example.com/products/10`

You can think of an endpoint as an API address.

For example:

`/products`

could represent all products.

And:

`/products/10`

could represent product number 10.

5. Foundational notes

The main idea today is very simple.

Suppose an API manages products.

The client might want to:

Read a product

Create a product

Update a product

Delete a product

REST-style APIs often represent those intentions using HTTP methods.

GET

POST

PUT

DELETE

Think of the HTTP method as the **verb**.

Think of the resource as the **thing being acted upon**.

For example:

GET `/products/10`

can be read approximately as:

GET product 10

Similarly:

DELETE `/products/10`

can be understood as:

DELETE product 10

This is a useful beginner mental model.

6. CRUD and HTTP methods

A very common software concept is **CRUD**.

CRUD stands for:

C → Create

R → Read

U → Update

D → Delete

These are four basic operations that many applications perform on data.

For example, with products:

Create a new product

Read an existing product

Update an existing product

Delete an existing product

REST APIs often connect CRUD operations with HTTP methods like this:

CRUD operation	HTTP method	Meaning
Create	POST	Create a new resource
Read	GET	Retrieve a resource
Update	PUT	Update a resource
Delete	DELETE	Delete a resource

A simple memory trick is:

CRUD

Create → POST

Read → GET

Update → PUT

Delete → DELETE

7. GET

GET is commonly used to **read or retrieve data**.

Example:

GET /products/10

Meaning:

"Give me product 10."

The server might return:

```
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

Conceptually:

```
Client
  |
  | GET /products/10
  ↓
Server
  |
  | Find product 10
  ↓
Product data
```

You can also imagine:

GET /products

meaning:

"Give me the products."

For today, remember:

GET → Read

8. POST

POST is commonly used to **create a new resource**.

Suppose the client wants to create a new product.

It might send:

POST /products

along with product information:

```
{
  "name": "Mouse",
  "price": 500
}
```

Conceptually:

```
Client
  |
  | POST /products
  | new product data
  ↓
Server
  |
  | Create product
  ↓
Response
```

The server might assign the product an ID:

```
{
  "id": 11,
  "name": "Mouse",
  "price": 500
}
```

For today, remember:

POST → Create

9. PUT

PUT is commonly used to **update an existing resource**.

Suppose product 10 currently contains:

```
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

You want to update it.

The request could conceptually be:

PUT /products/10

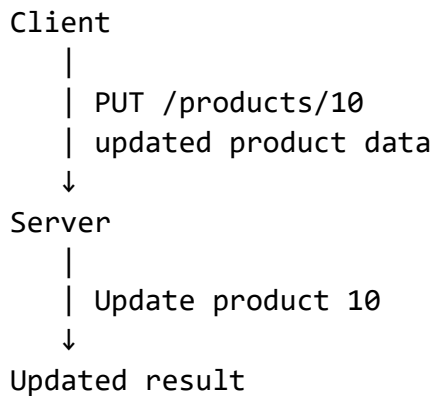
with new information:

```
{
  "name": "Mechanical Keyboard",
  "price": 1500
}
```

Meaning:

"Update product 10 with this information."

The flow looks like:



For today, remember:

PUT → Update

Do not worry yet about more detailed rules surrounding partial versus complete updates.

10. DELETE

DELETE is commonly used to **delete a resource**.

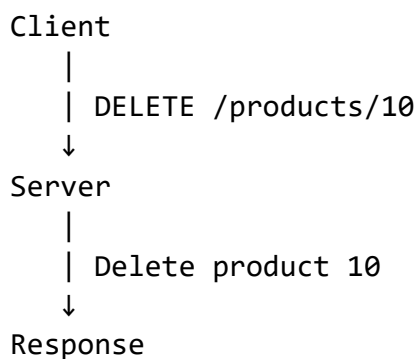
Example:

DELETE /products/10

Meaning:

"Delete product 10."

Conceptually:



The server may return a confirmation that the operation succeeded.

For today, remember:

DELETE → Delete

11. Easy products API example

Imagine an API with this base endpoint:

<https://api.example.com/products>

The server currently has:

Product 1 → Laptop

Product 2 → Keyboard

Product 3 → Mouse

Now let's perform different operations.

Read product 2

GET /products/2

Meaning:

Retrieve product 2.

Create a new product

POST /products

with:

```
{  
  "name": "Monitor",  
  "price": 8000  
}
```

Meaning:

Create a new product.

Update product 2

PUT /products/2

with:

```
{  
  "name": "Wireless Keyboard",  
  "price": 1500  
}
```

Meaning:

Update product 2.

Delete product 3

DELETE /products/3

Meaning:

Delete product 3.

12. Simple mapping

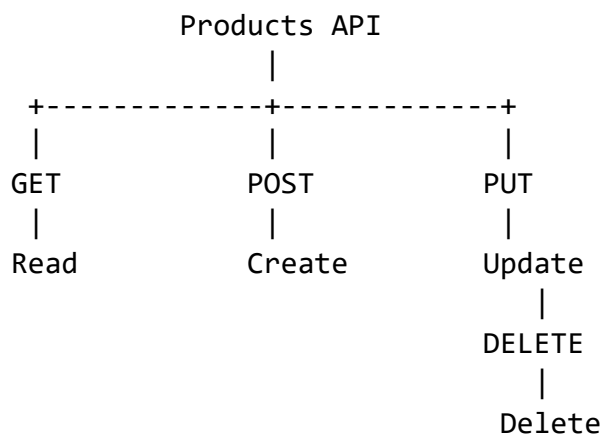
This is the main mapping to remember today:

GET → read product
POST → create product
PUT → update product
DELETE → delete product

Or with CRUD:

Create → POST
Read → GET
Update → PUT
Delete → DELETE

You can visualize the whole idea like this:



A cleaner version is:

```
Client wants to...
|
+-- Read -----> GET
|
+-- Create ----> POST
|
+-- Update ----> PUT
|
+-- Delete ----> DELETE
```

13. Resource + method thinking

When working with REST APIs, try to separate two questions.

First:

What resource am I working with?

Example:

product

Second:

What do I want to do with that resource?

Example:

read
create
update
delete

Then choose the HTTP method.

Example:

Resource:
product 8

Operation:
read

Method:
GET

Result:

GET /products/8

Another example:

Resource:
product 8

Operation:
delete

Method:
DELETE

Result:

DELETE /products/8

14. Problem statement

Suppose you have this products API:

<https://api.example.com/products>

Choose the correct HTTP method for each operation.

Operation 1

Retrieve product number 20.

/products/20

Which HTTP method should you use?

Operation 2

Create a new product:

```
{  
  "name": "Webcam",  
  "price": 2500  
}
```

Which HTTP method should you use?

Operation 3

Update product number 20 with a new price.

Which HTTP method should you use?

Operation 4

Remove product number 20.

Which HTTP method should you use?

Your job is only to identify:

GET
POST
PUT
or
DELETE

Do not write FastAPI or Python code.

15. Concepts used

This exercise uses:

REST API
Resource
Endpoint
HTTP method
GET
POST
PUT
DELETE
CRUD

The most important connection is:

Resource + desired operation



HTTP method

For example:

Product + Read



GET

16. Thought process

When you see an API operation, ask:

Step 1: What resource am I working with?

Example:

Product

Step 2: What does the client want to do?

Ask whether the operation is:

Create

Read

Update

Delete

Step 3: Map the operation to an HTTP method

Use:

Create → POST

Read → GET

Update → PUT

Delete → DELETE

Step 4: Identify the endpoint

For example:

/products

or:

/products/20

Step 5: Combine method and endpoint

Example:

Read product 20

becomes:

GET /products/20

You do not need to think about programming syntax yet.

17. Beginner-friendly pseudocode

Imagine a client wants to interact with products.

START

choose the product operation

IF operation is "read"
 use GET

ELSE IF operation is "create"
 use POST

ELSE IF operation is "update"
 use PUT

ELSE IF operation is "delete"
 use DELETE

send request to the products API

server processes the request

server sends a response

END

For one product:

IF user wants product information
 GET /products/product_id

IF user wants to update product
 PUT /products/product_id

IF user wants to delete product
 DELETE /products/product_id

For creating a product:

POST /products
send new product data

18. Suggested solving approach: REST-style thinking

Use this simple three-step approach.

Step A: Find the noun

The noun is usually the **resource**.

Example:

product

Step B: Find the action

Example:

retrieve

create

update

remove

Translate these into CRUD:

retrieve → Read

create → Create

update → Update

remove → Delete

Step C: Choose the method

Read → GET

Create → POST

Update → PUT

Delete → DELETE

Example:

"Remove product 5"

Resource:

product 5

Operation:

Delete

HTTP method:

DELETE

So conceptually:

DELETE /products/5

19. Easy edge case: product does not exist

Imagine the client sends:

GET /products/999

But product 999 does not exist.

The server cannot return that product.

Conceptually:

Client

|

| GET /products/999

↓

Server

|

| Search for product 999

|

| Product not found

↓

Error response

The important lesson is:

| Using the correct HTTP method does not guarantee that the requested resource exists.

The request itself may be logically correct, but the requested product may not exist.

You will learn proper HTTP error status codes later.

20. Easy edge case: invalid operation

Suppose someone says:

"I want to create a new product using GET."

That does not match the simple REST mapping you are learning.

For today's model:

GET → read

Creating a resource normally maps to:

POST

So first identify the actual intention:

Create new product

Then choose:

POST

Similarly:

Read product

should make you think:

GET

and not:

21. Quick comparison

Method	Main beginner meaning	Example
GET	Read	Get product 10
POST	Create	Add a new product
PUT	Update	Change product 10
DELETE	Delete	Remove product 10

A useful memory sentence is:

GET gets data.

POST creates data.

PUT updates data.

DELETE deletes data.

22. Hint only

For the problem, first translate each sentence into one CRUD word.

For example:

Retrieve → Read

Add → Create

Change → Update

Remove → Delete

Then use:

Create → ?

Read → ?

Update → ?

Delete → ?

Your four choices are:

GET

POST

PUT

DELETE

Do not think about code yet.

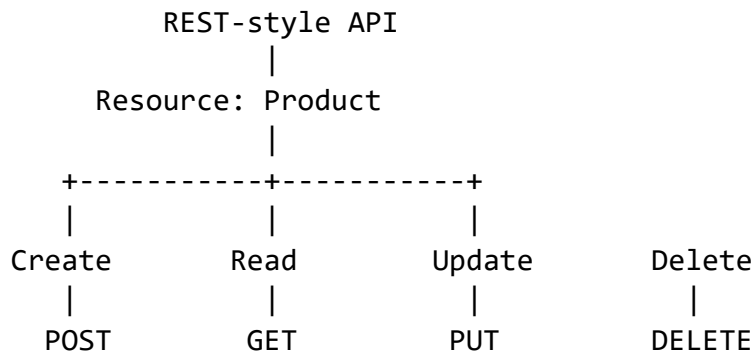
Day 122 key takeaway

Yesterday you learned:

Client → Request → Server
Client ← Response ← Server

Today you added the idea that the **HTTP method communicates the intention of the request.**

Remember:



The most important mapping for Day 122 is:

GET → Read
POST → Create
PUT → Update
DELETE → Delete

Once this feels natural, concepts such as API endpoints, request bodies, status codes, and FastAPI routes will be much easier to understand.